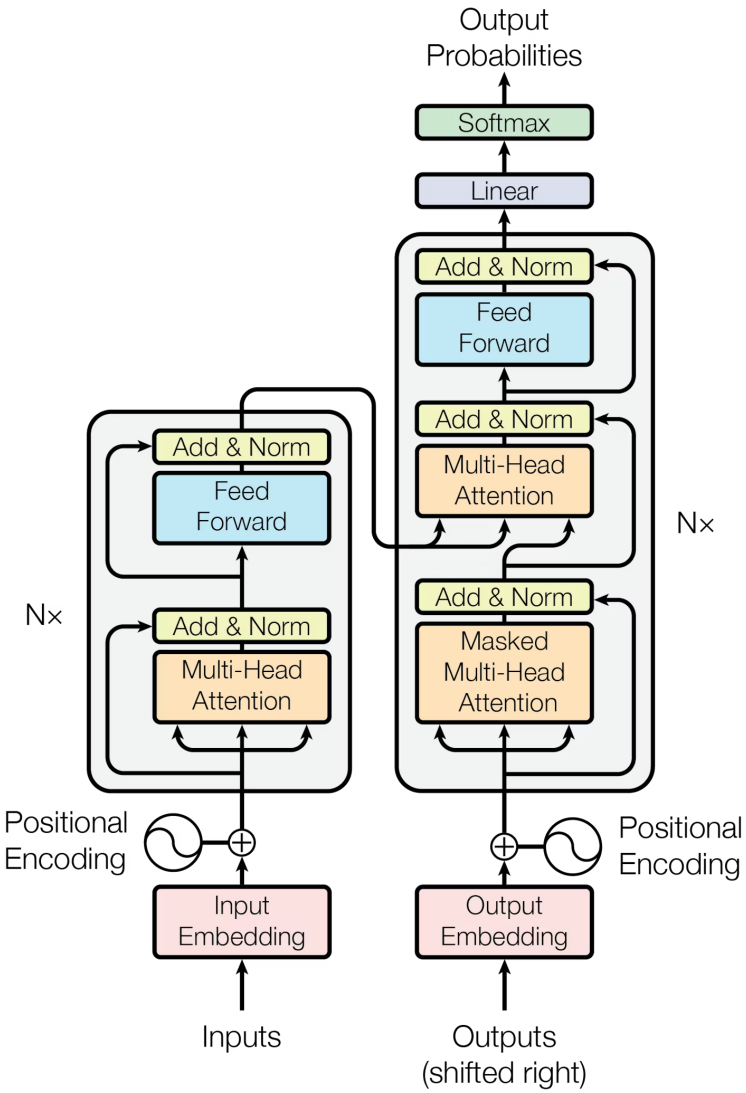
Transformer のお勉強

April 25, 2026

目次

1. **Transformer について**

Transformer



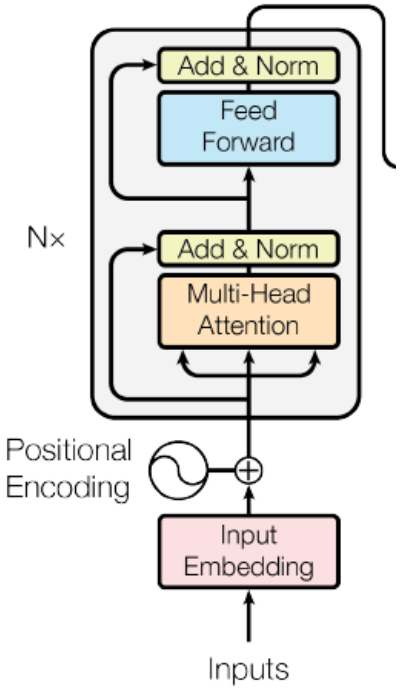
モデル

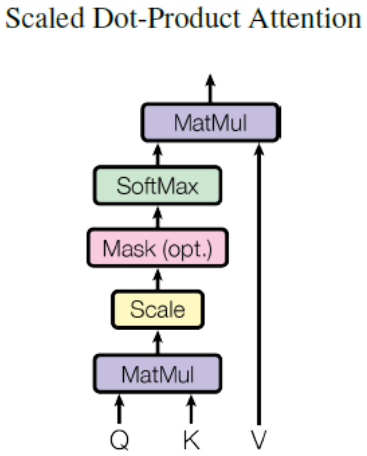
- RNN や CNN を使用せず、Attention のみで構成される
- 現行の LLM の多くが Transformer をベースとしている
- 翻訳タスクを解くモデルとして登場

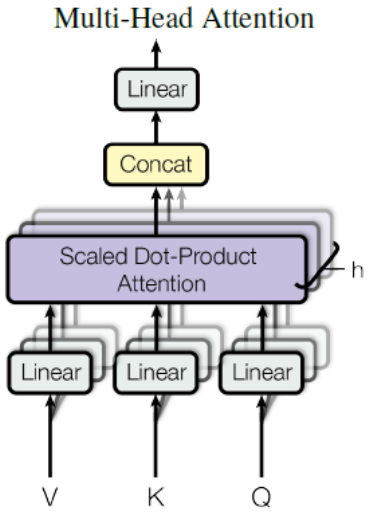
[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder



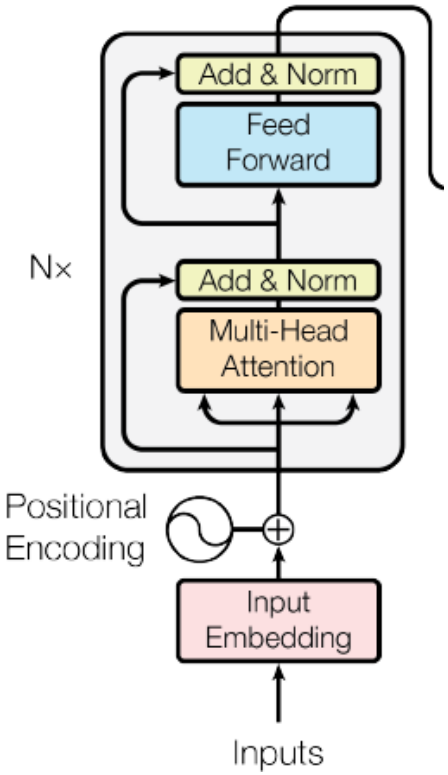




[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Input Enbedding



Input Embedding(埋め込み)

総トークン数を V として、V 次元の One-hot ベクトルを埋め込みに変換する行列 E を定義

$$E \in \mathbb{R}^{V \times d_{\text{model}}}$$

t=[0,0..,1,..,0,0] を, 第 k 成分のみ 1 で、他は全て 0 の One-hot ベクトルとする

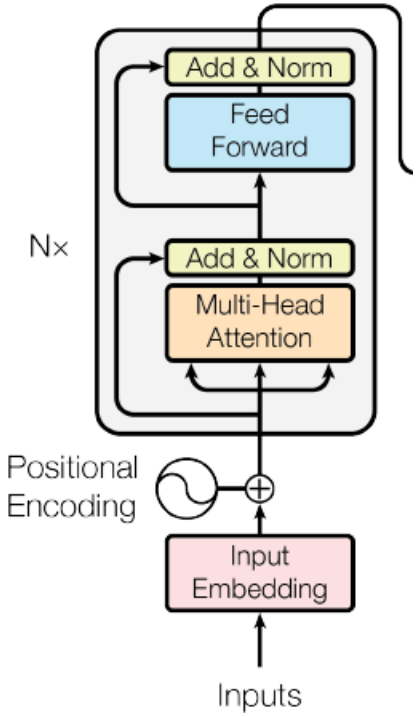
$$tE = d$$

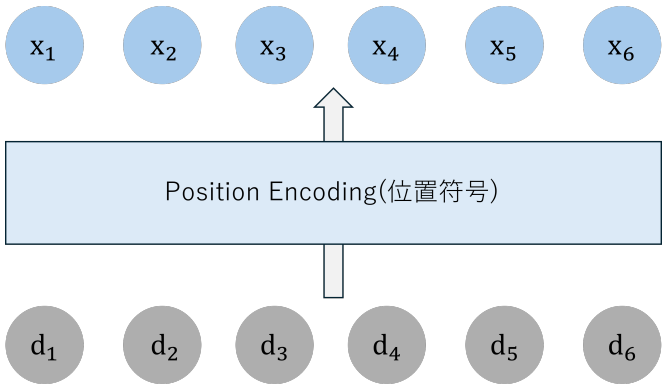
d は E の k 行目と一致する横ベクトル

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Positional Encoding





位置符号

入力埋め込みベクトル d に位置符号と呼ばれるベクトルを加算
→語順の情報が付与される

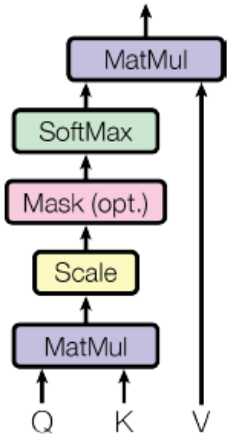
正弦関数と余弦関数を用いて位置ごとに異なる固有のパターンをもつベクトルを生成し、d に直接加算するらしい

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Scaled Dot-Product Attention

Scaled Dot-Product Attention



$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

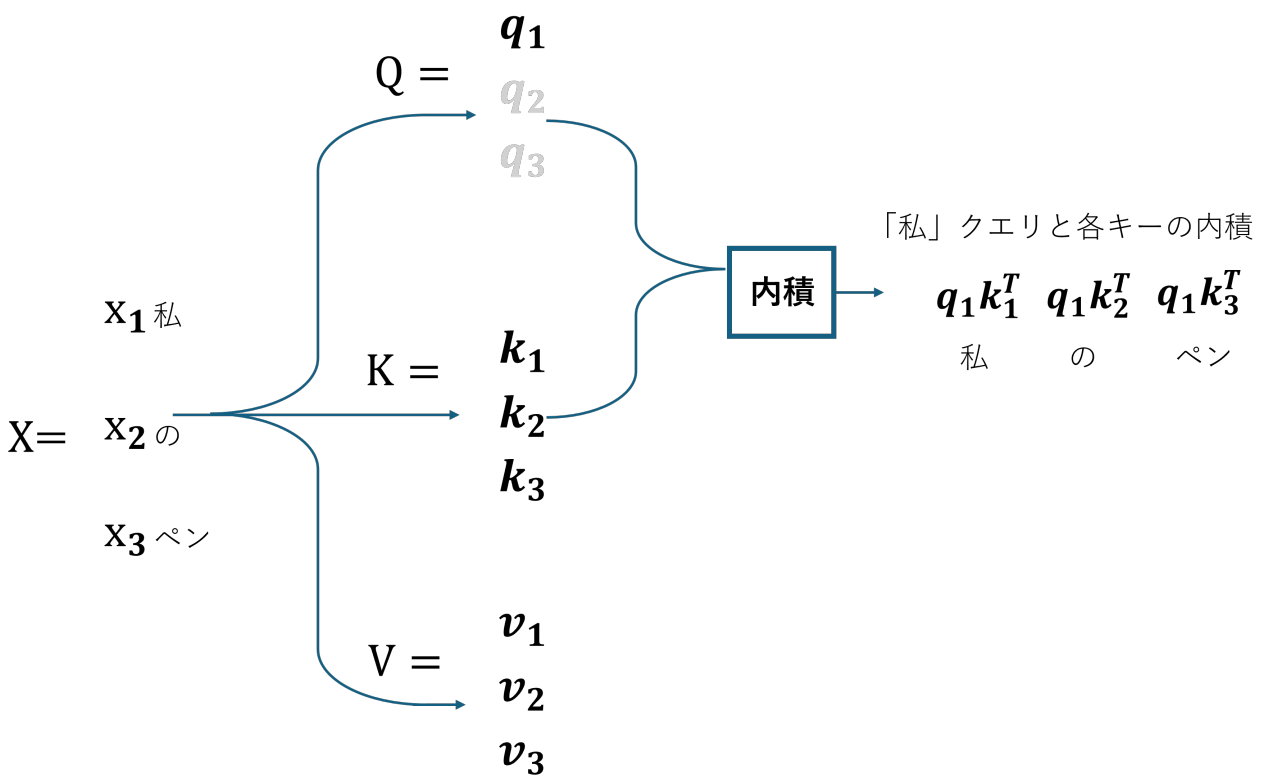
処理フロー

- **1. MatMul**: Q と K の内積により単語間の相関（スコア）を算出
- **2. Scale**: $\sqrt{d_k}$ で除算。d_k 増大による勾配消失を防止
- **3. Softmax**: スコアを合計 1 の確率（重み）に正規化
- **4. MatMul**: 重みに基づき V を足し合わせ、文脈を集約

変数	名称	イメージ・定義
$Q \in \mathbb{R}^{n \times d_k}$	Query	「何に注目したいか」
$K \in \mathbb{R}^{n \times d_k}$	Key	「どのような情報を持っているか」
$V \in \mathbb{R}^{n \times d_v}$	Value	実際に抽出される「情報」
n	シーケンス長	入力トークン数
d_k	次元数	Q, K の次元数
d_v	次元数	V の次元数
softmax	関数	正規化する処理

Transformer

Encoder - Scaled Dot-Product Attention

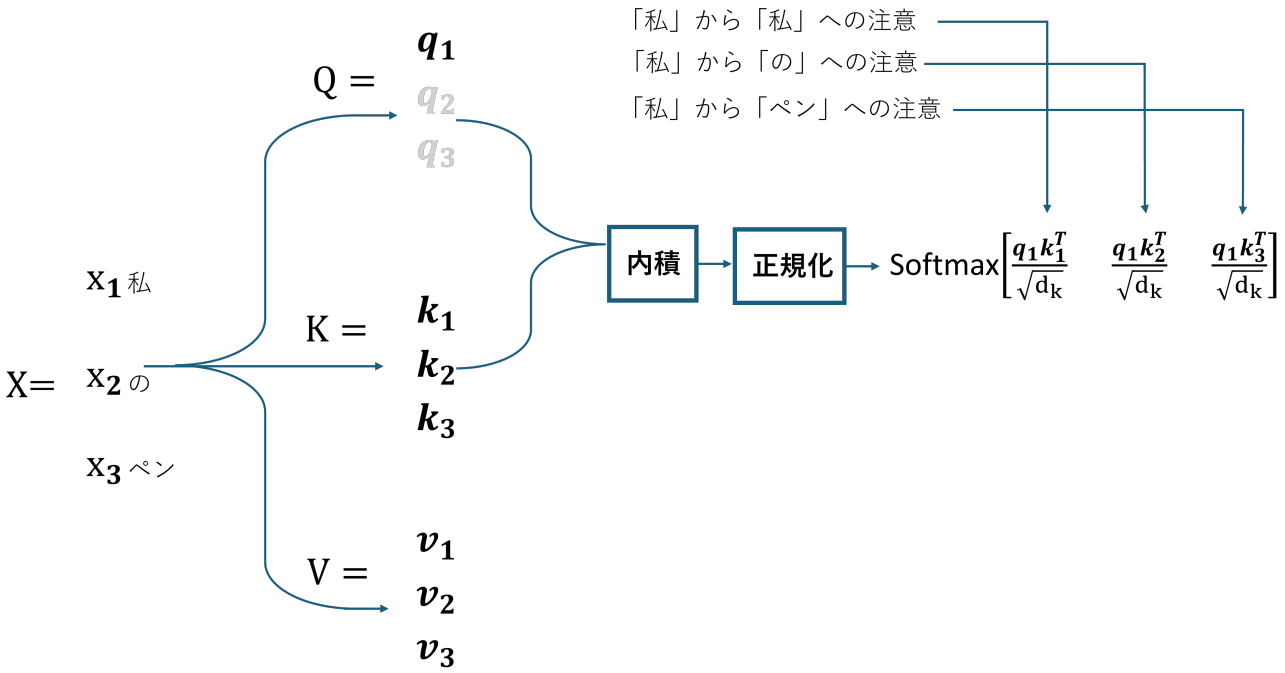


Query(注目したい単語) と各 Key(文中の各単語) がどれだけ関連しているか (類似度) を算出

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Scaled Dot-Product Attention

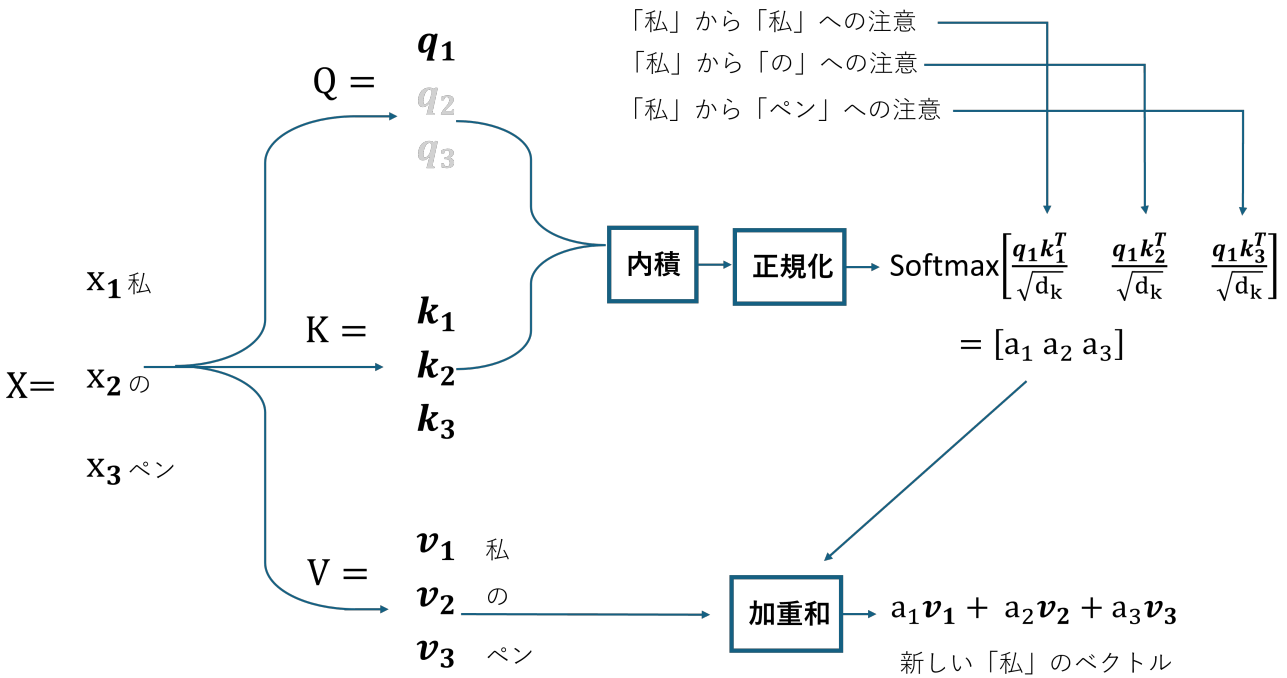


Query と各 Key の内積を正規化したものを「対象の単語が周りの各単語をどれほど注視しているか」の指標とみなす (=注意重み)

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Scaled Dot-Product Attention

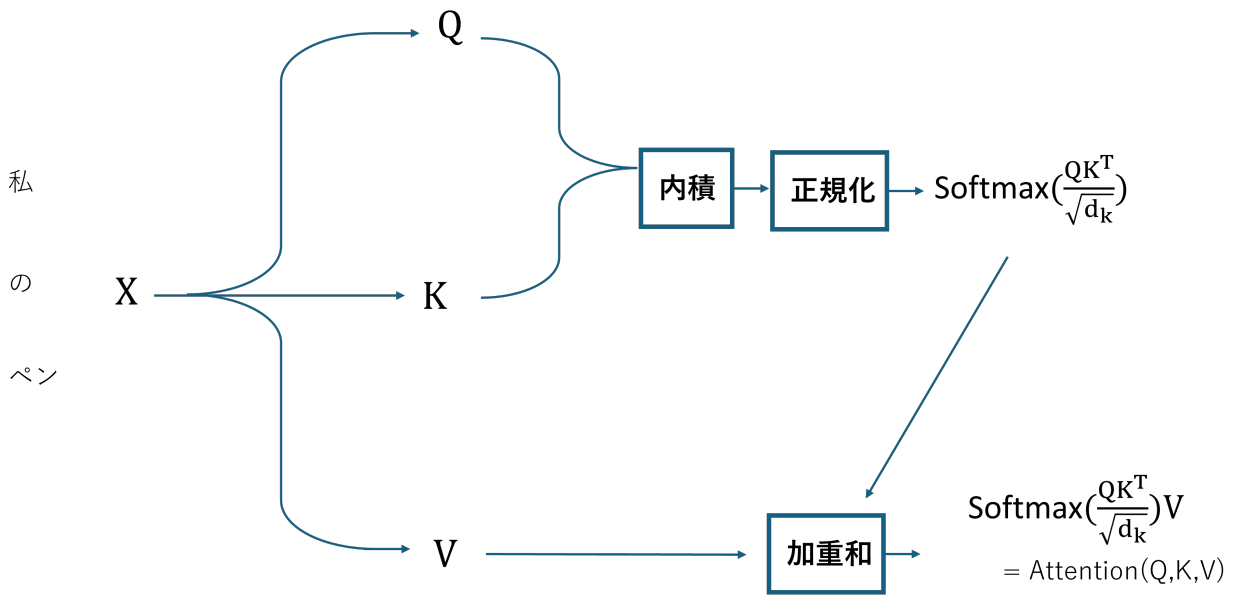


注意重みで重み付けされた Value の和を計算することで「私」のベクトルに周辺の単語情報が付加される

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Scaled Dot-Product Attention

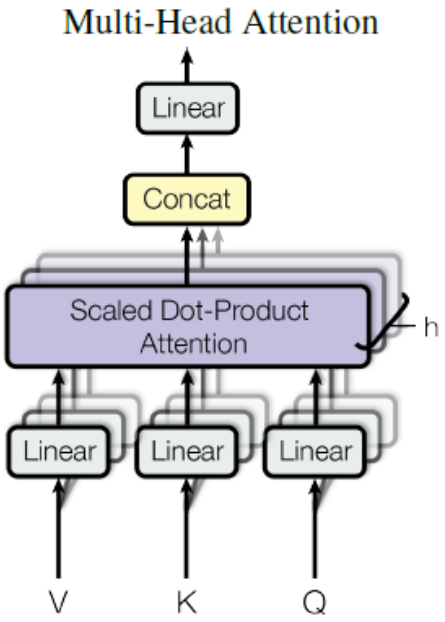


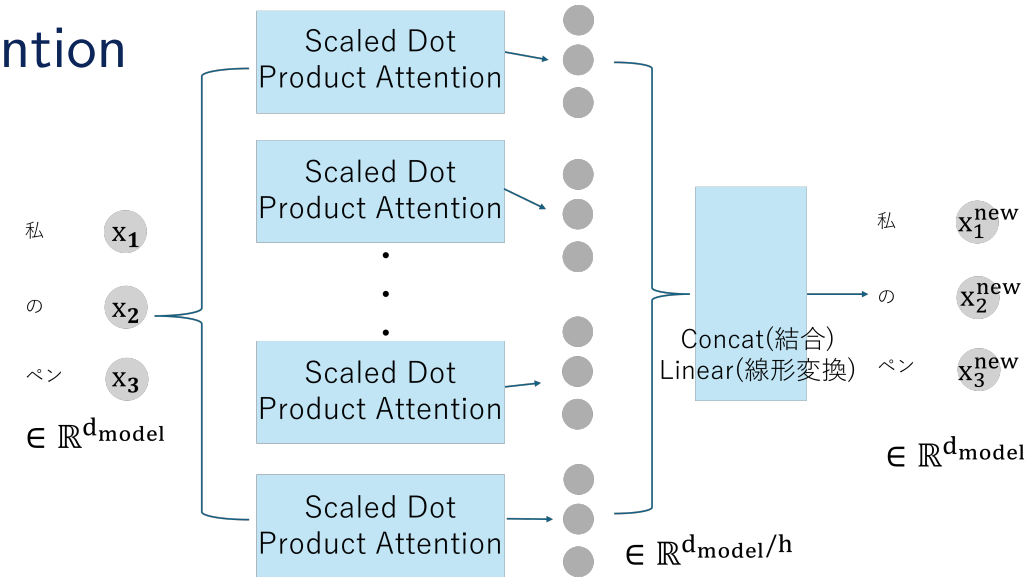
x_1、x_2、x_3 の更新は独立しているため並列化可能
$\rightarrow x_1$ を行列 X の 1 列目の要素のようにみなすと X 全体の更新を Attention(Q,K,V) で行える

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Multi-Head Attention





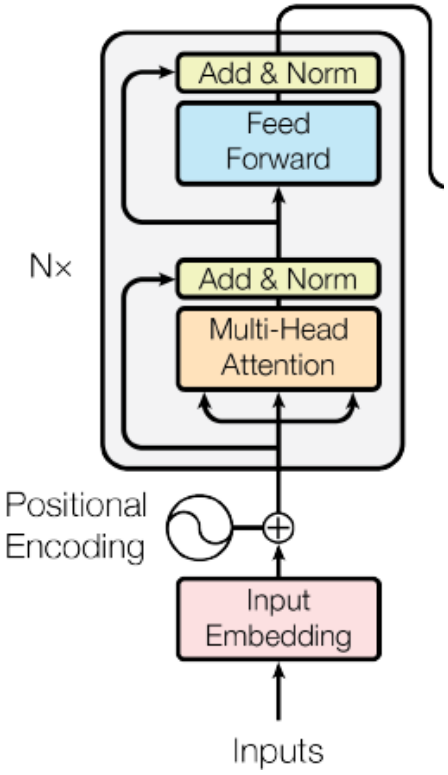
$$\text{Multi Head}(Q, K, V) = \text{Concat}(\text{head}_1, , , \text{head}_h)W^o$$
$$\text{head}_i = \text{Attentio}(QW_i^Q, KW_i^K, VW_i^V)$$

変数	名称
$W^Q, W^K \in \mathbb{R}^{d_{model} \times d_k}$	Q, K の投影用の重み
$W^V \in \mathbb{R}^{d_{model} \times d_v}$	V の投影用の重み
$W^O \in \mathbb{R}^{hd_v \times d_{model}}$	出力統合用の重み
d_k, d_v	各ヘッドの次元数
d_{model}	埋め込みベクトルの次元数
h	ヘッド数

[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Add & Norm



Residual Connection & Layer Norm

各サブレイヤの出力を以下の処理を行う

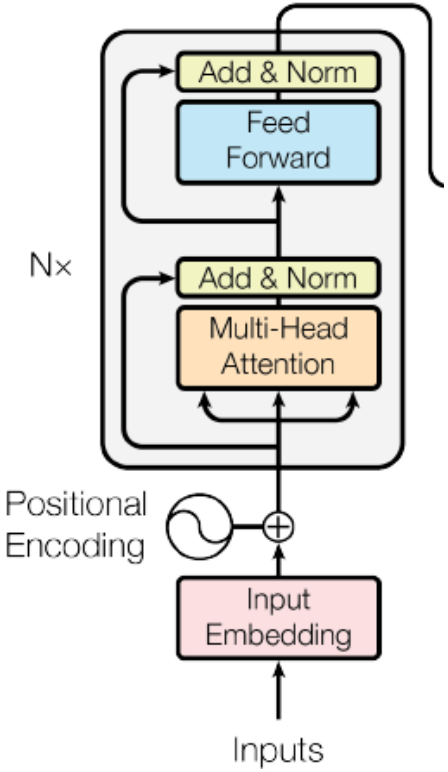
$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- **Add (Residual)**: 前の層の出力 $\text{Sublayer}(x)$ へ入力 x を直接足す
  →勾配消失を防ぎ、深い層でも学習が安定
- **LayerNorm**: 特徴ベクトルの平均を 0、分散を 1 に正規化
  →学習が高速化・安定化

[7] Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Feed Forward Network



Position-wise Feed Forward (FFN)

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

変数・次元数定義：

- $W_1 \in \mathbb{R}^{d_{\mathrm{model}} \times d_{ff}}$ (512 → 2048)
- $W_2 \in \mathbb{R}^{d_{ff} \times d_{\mathrm{model}}}$ (2048 → 512)

Attention 層から出力された関係性情報を、非線形変換し、より抽象度の高い特徴を抽出する

[7] Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

参考文献

[1] Warstadt, A. et al., Findings of the BabyLM Challenge: Sample-Efficient Pretraining on Developmentally Plausible Corpora, Association for Computational Linguistics, 2023.

[2] Kosmopoulou, D. et al., Masked Diffusion Language Models with Frequency-Informed Training, Proceedings of the First BabyLM Workshop, Association for Computational Linguistics, 2025.

[3] Păpușoi, R. et al., A Comparison of Elementary Baselines for BabyLM, Proceedings of the First BabyLM Workshop, Association for Computational Linguistics, 2025.

[4] Ueda, R. et al., Syntactic Learnability of Echo State Neural Language Models at Scale, Proceedings of NAACL 2025, 2025.

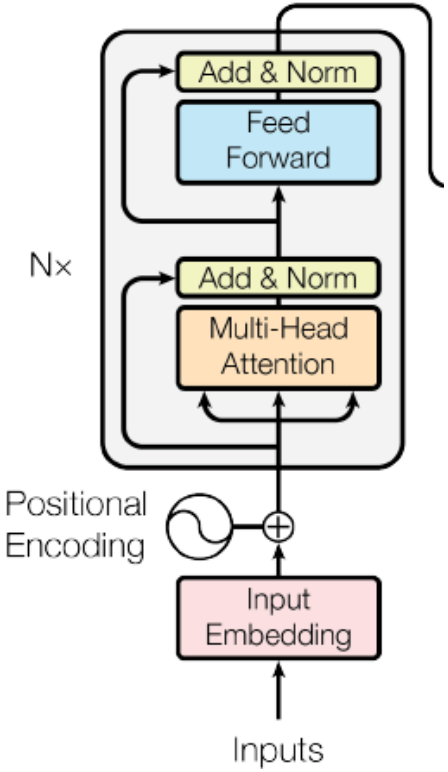
[5] Warstadt, A. et al., BLiMP: The Benchmark of Linguistic Minimal Pairs for English, MIT Press, 2020.

[6] Mikolov, T. et al., Efficient Estimation of Word Representations in Vector Space, Proceedings of ICLR, 2013.

[7] Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

Transformer

Encoder - Positional Encoding



Positional Encoding(位置エンコーディング)

単語の順序情報を付与するため、埋め込みベクトルに加算する。

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

変数定義：

- pos: 文中でのトークンの位置（0, 1, ...）
- i: 次元のインデックス（0 から $d_{\text{model}}/2$ まで）
- d_{model}: モデルの次元数（512）

役割：RNN のような逐次処理を行わないため、絶対的・相対的な位置関係を正弦波の周期性により表現する。